IntelliERP: Verified, Grounded LLM Decision Support for Cross-Module ERP Operations with Human-Approved Actions

Author Name
Department
University
City, Country
email@university.edu

Abstract—Large language models (LLMs) are being added to enterprise resource planning (ERP) systems as natural-language assistants, but their statements about stock, demand and suppliers are rarely checked, and the step from an AI recommendation to an ERP transaction is rarely controlled. We present IntelliERP, an ERP decision-support system in which deterministic analytics decide what is at risk and the LLM only explains and recommends; every figure and every qualitative claim the model makes is checked against the system's data before the user sees it; and the only change the AI can make is a draft purchase order that passes the same validation as any order and must be approved by a second person. The assistant answers by calling 15 permission-checked tools and is grounded in a 41-passage knowledge base of the company's schema, definitions and rules. On a wholesale dataset derived from Microsoft's Wide World Importers (227 products, 72,696 sales lines), we evaluated 86 questions in three repeated runs with keyword- and number-based scoring and no model as judge. The system passed 84, 86 and 86 of 86 questions; on 32 paired questions it passed 32 of 32 with grounding and 8 to 9 without ($p < 0.0001$, exact sign test). A first run exposed five failures that led to design changes, and 84 of 86 questions then passed in every repeat. A claim verifier covering 12 kinds of statement, frozen before testing, detected 48 of 62 (77%) model-injected false statements within its scope on two unseen test sets, raised false alarms on 2 of 124 paraphrases (2%), and found genuine model errors that keyword scoring had passed.

Index Terms—enterprise resource planning, large language models, decision support, grounding, hallucination, human-in-the-loop, supply chain

I. Introduction

ERP systems hold the data managers need to see an operational problem coming: sales show demand, inventory shows stock, and procurement shows what is on order and how reliable each supplier is. The signals are spread across modules, so recognising, for example, that a product will run out before its next delivery means reading several screens and applying the company's own rules. LLM assistants promise to do this from a question in plain language, and research on AI in ERP is growing by 11.3% a year [1]. Three problems stand in the way.

Accuracy on enterprise data. LLMs that write SQL over realistic enterprise schemas solve as few as 21% of tasks [2], and business context is the main lever for improving them [3], [4].

Unchecked statements. A fluent answer can state a wrong trend or risk level. Agentic inventory systems claim explainability without checking the explanations [5]; the one system we found that checks them does so only for their direction, in a single case study [6].

Uncontrolled actions. Autonomous ordering agents can cut cost on average yet vary widely from run to run [7], and some authors argue for removing the human from the loop altogether [8]. Governance and trust appear in under 2% of AI-ERP studies [1].

We address these with IntelliERP and make four contributions:

1) An architecture in which rule-based analytics decide what is at risk and the LLM only explains and recommends, through permission-checked tools, grounded in a knowledge base generated partly from the system's own code so it cannot drift from it.
2) Run-time verification of the model's output: every figure is looked up in the data, and every qualitative claim (below safety stock, demand rising, a delivery due in time, a stock status, a risk level, and seven other kinds) is checked against the same facts; contradicted output is returned to the model to be corrected, or flagged to the user.
3) A controlled path from recommendation to transaction: the AI can create only a draft purchase order, which passes the same validation as any order, records its AI origin and evidence, and needs approval by a different user.
4) An evaluation with automatically computed answer keys, three repeated runs, an ablation of grounding, and held-out tests of the claim verifier on model-written perturbations, with the verifier frozen before each test.

II. Related Work

LLMs over ERP and enterprise data. The closest system, "Chatting with your ERP" [9], uses a reasoner and a critic to write SQL over a production ERP database; it is read-only and depends on a hand-written description of the database. Benchmarks show that direct

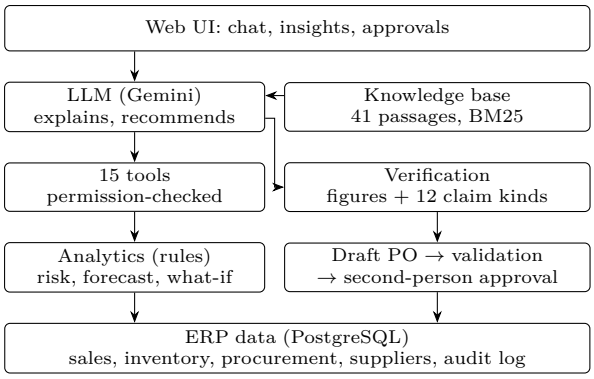


Fig. 1. IntelliERP. The analytics decide what is at risk; the LLM explains through tools; its output is verified; its only action is a draft that a second person approves.

SQL generation remains the weak point [2], [4] and that questions often need reasoning beyond relational queries [10]. Supplying business context as a knowledge graph tripled accuracy in [3]. IntelliERP avoids SQL generation: the model calls tools that wrap the application's services.

LLMs in supply chains. Reviews describe the field as fragmented and lacking deployment frameworks [11]. OptiGuide [12] keeps a solver in charge of numbers while the LLM explains, the division IntelliERP also follows. Multi-agent systems let LLMs place orders directly [5], [13], and studies of such agents report unstable decisions across runs that guardrails reduce [7]. Industrial planning agents remain advisory [14].

Agents and tool use. IntelliERP's assistant follows the reason-and-act pattern [15], [16]. Enterprise benchmarks find agents below 58–65% success with poor rule following [17], a large gap to automation [18], and low reliability across repeated trials, measured as pass^k [19].

Verification and abstention. Self-RAG [20] and Chain-of-Verification [21] let a model critique its own output; FActScore [22] scores atomic facts against a source; RAGAs [23] uses an LLM as judge. IntelliERP's checks are deterministic and run against the system's own data, not the model's judgement. Declining to answer is surveyed in [24].

Safety and proactive assistance. Indirect prompt injection [25] and its benchmarks [26] motivate limiting what an agent may do. Proactive agents learn when to raise an issue [27], and InsightBench [28] evaluates business-insight generation. Human-AI design guidelines ask that a system make clear why it acted [29].

III. System Architecture

Fig. 1 shows the system. A React web application calls a FastAPI back end over REST; data is stored in PostgreSQL behind repository interfaces. Role-based access control (four roles) is enforced on every request.

Analytics. Deterministic functions compute, per product, average daily demand over 28 days and its trend; days of cover; lead time observed from deliveries; a stockout-risk level (Critical, High, Medium, Low) from fixed rules combining stock, safety stock, demand trend, lead time and deliveries due; a reorder quantity from an order-up-to rule; weekly forecasts (moving average or linear trend, chosen by backtest error); supplier on-time rate, delays and delay trend; and day-by-day what-if projections. Each result carries the facts that produced it.

Assistant. For each question the model is offered only the tools the user's role permits; the permission is checked again when a tool runs. The tools wrap the same services as the web pages, so the model never writes SQL and takes every figure from a tool result. One tool returns a product's facts grouped by module (sales, inventory, procurement, supplier, assessment). The user's role, what it excludes and what it allows with purchase orders are stated with each question. List tools return totals over all matching rows and say when rows were cut off.

Grounding. A knowledge base of 41 passages describes every table and column (11, generated from the database models), definitions (11), rules (8), workflows (4), role permissions (4, generated from the permission map) and data notes (3). Thresholds in written passages are filled from the constants of the code that applies them, and a test fails if a column lacks a description. The four passages best matching a question (BM25 over title, keywords and text) are placed before it; the model can search for others.

Proactive insights. A scheduled scan takes every High or Critical finding from the analytics, gives the model the cross-module facts and applicable rules for the most severe six, and records one insight per finding with a recommended action. The model cannot raise an insight about anything not flagged.

Controlled actions. A reorder recommendation can become a draft purchase order, by a button or on request in the chat. The draft passes the validation every order passes (active supplier that supplies the product, quantity limits), records its AI origin and the recommendation with its checks, and must be submitted by a user with the right and approved by a different user. The assistant has no tool that submits, approves or places an order. If the server refuses a draft from a supplier the user named, a draft from a supplier the user did not name is refused for the rest of that message.

IV. Run-Time Verification of Model Output

Figures. Every number in an answer, recommendation or insight is looked up among the tool results, the knowledge passages supplied, the question and earlier turns, allowing for rounding; identifiers and dates are ignored. An answer that states figures although no tool was called is returned to the model once with a request to call a tool or drop them. Figures that still cannot be traced are returned to the user as unverified. A figure derived correctly by arithmetic is flagged, not blocked.

TABLE I
Claims Verified and the Fact Each Is Checked Against

Claim	Checked against
below / above safety stock	stock on hand vs. safety stock
out of stock / in stock	stock on hand $= 0$ / > 0
demand rising / falling	28-day demand trend > 0 / < 0
units overdue / on time	overdue units or orders
delivery in time / too late	next delivery vs. stockout date
cover below / above lead time	days of cover vs. lead time
below / above reorder level	stock on hand vs. reorder level
below / above reorder point	stock + on order vs. reorder point
stock status	status rule on the stock record
stockout risk level	assessed risk level
supplier delivery risk level	supplier risk level
supplier delay trend	worsening / improving / stable

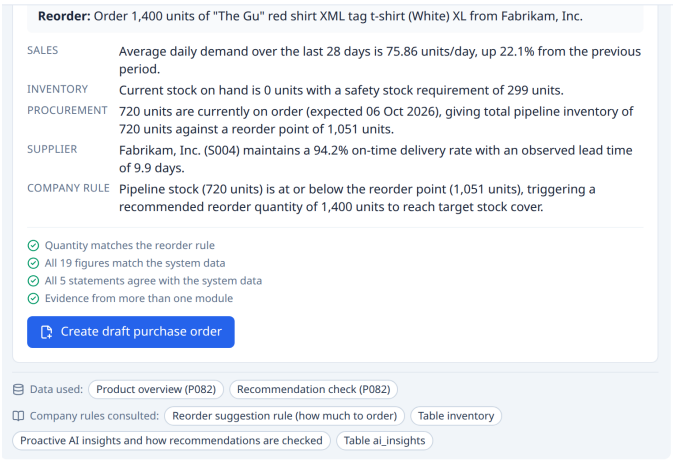


Fig. 2. A reorder recommendation in the assistant (lower part of the card): evidence from each module, the company rule applied, the results of the checks, the only action offered (a draft), and the tools and knowledge passages used.

Claims. A real figure can still be used to say something false, and many statements carry no figure. The claim verifier finds 12 kinds of qualitative statement (Table I) by pattern, attributes each sentence to the product it names (or the one named most recently; sentences naming several products are skipped), and judges each claim against the same facts the model was given. A claim is supported, contradicted, or unverifiable when the needed fact is missing (e.g. no demand trend for a product with too few orders). Negations ("not", "no", "none", "0 units") reverse a claim within their clause; rule descriptions and hypotheticals ("Medium risk applies if…", "would run out on…") and list headings are not treated as claims.

What happens to a contradiction depends on where it occurs: a chat answer is returned to the model once with the contradicted statements and what the data says; a recommendation is returned with an instruction to record a corrected one, which replaces it; an insight is not stored until corrected; a draft purchase order is flagged for its reviewer and approver. Remaining contradictions are shown to the user. The checks are deterministic and need no model call. Fig. 2 shows a recommendation with its checks.

V. Evaluation Setup

Data. Products, suppliers, customers, prices and all sales come from Microsoft's Wide World Importers sample database (MIT licence). Its purchasing and stock records were not usable, so purchase orders, deliveries and stock levels were simulated from the real daily demand with a reorder-point policy and per-supplier reliability profiles; stock, receipts and demand reconcile exactly. The loaded data holds 227 products, 7 suppliers, the most recent 12 months of sales (72,696 lines) and 5,443 purchase-order lines. Supply-side results are therefore on simulated data.

Questions. 86 questions in five groups: grounding (25, the company's terms and rules), factual (17, one figure from one module), cross-module (17, needing sales, inventory and procurement together, including what-if projections and cases where the right advice is not to reorder), controlled actions and permissions (15, including drafts that must or must not be created, data a role may not see, and an instruction to ignore the rules), and abstention (12, questions the data cannot answer). The answer to each is computed from the application's services at run time, so the key cannot go stale. 32 questions are also asked with the knowledge base switched off.

Scoring. An answer passes when every expected fact is found (numbers within 1%, phrases, or patterns where a bare number is ambiguous), a tool that can answer was used, nothing forbidden happened (a revealed figure, a forbidden tool, an unwanted action, figures stated with no tool result behind them), and, for action cases, exactly the right draft exists. No model is used as a judge. Keys were corrected after reading answers in eleven cases (four correct answers worded unexpectedly, one using an unlisted tool, seven wrong answers that had passed; one key was corrected in both directions), applied to all runs and conditions and documented with the results.

Runs. All answers came from gemini-3.5-flash-lite with no fallback model. Run E1 used the system as first built; after it, five failures led to changes (Section VI-C); runs F1–F3 repeat the same questions on the changed system, which was not altered between them. Pass rates have Wilson 95% intervals [30]; grounding is compared with the exact sign test on paired questions.

VI. Results

A. Overall and per group

Table II gives the results. After the changes the system passed 84, 86 and 86 of 86 questions with grounding; 84 questions passed in all three runs and none failed in all three. With grounding, the figures not found in the data numbered 1 of 310, 3 of 342 and 2 of 332, all derived correctly from tool results (a difference of two projections, a lead time plus a 7-day buffer, 0 units in a warehouse that does not exist).

TABLE II
Questions Passed With Grounding (of 86)

Group (questions)	E1	F1	F2	F3
Grounding (25)	25	25	25	25
Factual (17)	16	17	17	17
Cross-module (17)	17	16	17	17
Actions and permissions (15)	12	15	15	15
Abstention (12)	11	11	12	12
All (86)	81	84	86	86
95% CI (%)	87–97	92–99	96–100	96–100



Fig. 3. Claim verifier by version, with Wilson 95% intervals. v1 is an estimate from a reviewed sample of mutation-test misses. False alarms on paraphrases: v2 1 of 60, v3 2 of 124.

B. Effect of grounding

On the 32 paired questions the system passed all 32 with the knowledge base in every run, against 9, 8, 9 and 9 without it; in each run 23 or 24 questions passed only with grounding and none only without ($p < 0.0001$). Without grounding it still answered 5 or 6 of the 7 paired cross-module questions, because the tools return those facts, but only 2 to 4 of the 25 questions on the company's terms and rules: it cannot know the thresholds, and several answers supplied plausible thresholds of its own or stated that the simulated supply data was real.

C. Failures and the changes they led to

Run E1 exposed five failures, each a different weakness: (i) counting overdue orders in the 20 rows a list tool returned out of 434, giving 1 instead of 7; (ii) declaring a real order missing for the same reason; (iii) after correctly refusing a request, listing product figures and a supplier that no tool had returned; (iv) announcing a draft without creating it and telling a procurement manager they could not submit orders; (v) after the server refused a draft from a supplier that does not supply the product, drafting from another supplier without asking. The changes were list totals and truncation notes, lookup of an order by its ID, the no-source figure correction, stating the user's rights, and refusing substitute suppliers. All five questions passed in all three later runs; since each change was written with its question in view, this is weak evidence for the changes in general. Two other questions failed once in F1 and passed twice.

D. Proactive insights

The 18 insights produced by three scans on the current data all concern a finding the analytics flagged (enforced), all cite evidence from at least two modules, all 266 figures in them are found in the data, and all three reorder insights carry the quantity the reorder rule gives.

E. Claim verification

Method. The verifier was developed in three versions. Each held-out test used texts whose wording had not been seen while that version was written, and the verifier's source file was frozen (its hash recorded) before the test was scored. Test texts are a seeded random sample of stored answers and insights that name a product; the selection does not depend on the verifier, so successive versions were tested on successive, disjoint blocks of texts: version 2 on the first 60, version 3 on the next 60 and, unchanged, on the remaining 64. For each text the model (gemini-3.5-flash-lite, five texts per request) produced two rewrites: one with exactly one qualitative statement made false without changing any number, and one paraphrase with the same meaning. An injection counts when its stated change is visible in the text; only contradictions the rewrite introduced count as detections. An injected error is in scope when it states something about a product's or supplier's current state of one of the verifier's claim kinds; the rest change permissions, whether something exists, comparisons between products, past events, periods, scenario premises, process statements or numbers (the figure check's domain). Scope was classified by hand and every classification is published with the results.

Development. On the stored answers and insights (558 claims for the final version) an early version raised 45 flags, all false alarms (unrecognised negations, rule descriptions, projections, headings, supplier and stockout risk confused); in a manual check of 40 claims it judged supported, 38 were correct and 2 correct only by coincidence. A word-swap mutation test (271 mutants, 40 misses reviewed by hand) estimated that version 1 caught about 48% of real errors and showed why: no rules for "on time", "in stock", reorder levels or points, and two over-broad filters.

Held-out results. Table III gives the results. Version 2 caught 15 of 25 in-scope errors; its misses revealed a defect (a decimal point inside a sentence ended a pattern's search) and unrecognised phrasings, which version 3 fixed. Version 3 caught 25 of 31 in-scope errors on its first test set and 23 of 31 on the second, 48 of 62 in all (77%, Fig. 3). Its 14 misses were unrecognised phrasings (e.g. "arriving on time", "in danger of running out", "significantly higher than the lead time", a risk level written as a list field); its two false alarms, both in the first set, were a six-word list heading read as a claim and an unknown negation ("stopping the item from hitting Critical risk"). None was fixed, so each test remains a test.

Errors missed by the evaluation. On the stored answers the final verifier flags four statements, in two answers, and

TABLE III
Claim Verifier on Held-Out Model-Written Rewrites

	v2	v3, set 1	v3, set 2
Texts	60	60	64
Valid injected errors	59	60	64
In scope	25	31	31
Detected, in scope	15 (60%)	25 (81%)	23 (74%)
95% CI	41–77%	64–91%	57–86%
Detected, all	16 (27%)	25 (42%)	23 (36%)
False alarms (paraphrases)	1 of 60	2 of 60	0 of 64

all are genuine model errors that the keyword evaluation had passed: in two runs the assistant said a product with 125 units and a reorder level of 92 had status "Low Stock" because "125 units are below the reorder level of 92". Its status is In Stock. The rule that caught this was added after the mutation review, which included this sentence.

VII. Discussion and Limitations

Grounding mattered most for the company's own terms and rules, which a general model cannot know; for cross-module facts the tools carried the answer. Keeping detection in deterministic analytics and limiting the model to explanation removed the run-to-run variation that autonomous ordering agents show [7] from the decision of what is at risk. Verification adds little cost: it is pattern and lookup based and needs no model call. It complements keyword scoring: it found errors that answers containing the expected facts still made.

The results have clear limits. The questions were written by the system's builders and favour what the system covers. Scoring by keyword needed corrections after reading answers. All answers came from one small model. Supply-side data is simulated. The figure check cannot tell a correct derived figure from an invented one. The claim verifier checks only the 12 kinds of statement it recognises, by pattern, in English; its tests used errors injected by a model, whose style may differ from errors a model makes on its own, and even 62 in-scope errors leave an interval of 66–86%. Scope was judged by the authors. No study with real users was made, so usefulness to managers is not measured.

VIII. Conclusion

IntelliERP shows that an LLM can be made a useful ERP assistant without being trusted blindly: the analytics decide what is wrong, the model explains through permission-checked tools, its figures and claims are checked against the data before anyone sees them, and its only action is a draft a second person must approve. Grounding raised paired-question accuracy from 8–9 to 32 of 32, the repeated runs were stable, and the claim verifier caught 77% of in-scope false statements on unseen text with 2% false alarms. Future work: questions written by others, other models, a study with real users, broader claim coverage, and a test of indirect prompt injection through ERP text fields.

References

[1] L. Hurbean, S. C. Necula, D. Fotache, and I. M. Stepan, "Unveiling AI-powered ERP systems: a systematic review of trends and future directions," Information Systems and e-Business Management, 2026.

[2] F. Lei et al., "Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows," in International Conference on Learning Representations (ICLR), 2025.

[3] J. Sequeda, D. Allemang, and B. Jacob, "A benchmark to understand the role of knowledge graphs on large language model's accuracy for question answering on enterprise SQL databases," 2023.

[4] J. Li et al., "Can LLM already serve as a database interface? A BIg bench for large-scale database grounded text-to-SQLs," in Advances in Neural Information Processing Systems (NeurIPS), 2023.

[5] Y. Quan and Z. Liu, "InvAgent: A large language model based multi-agent system for inventory management in supply chains," 2024.

[6] Z. Xue, Y. Wang, and M. Huo, "LLM-grounded explainable AI for supply chain risk early warning via temporal graph attention networks," 2026.

[7] C. X. Long, D. Simchi-Levi, F. Zhu, H. Su, A. P. Calmon, and F. P. Calmon, "Reliability and effectiveness of autonomous AI agents in supply chain management," 2026.

[8] D. Simchi-Levi, K. Mellou, I. Menache, and J. Pathuri, "Large language models for supply chain decisions," 2025.

[9] J. Ruiz Gómez, L. Andrés Susinos, J. Alamo Olivé, S. Rey Osorno, and M. L. González Hernández, "Chatting with your ERP: A recipe," 2025.

[10] A. Biswal, L. Patel, S. Jha, A. Kamsetty, S. Liu, J. E. Gonzalez, C. Guestrin, and M. Zaharia, "Text2SQL is not enough: Unifying AI and databases with TAG," in Conference on Innovative Data Systems Research (CIDR), 2025.

[11] Z. Song, Y. Xie, L. Yang, and Y. Zhao, "Large language models in supply chain management: a systematic literature review and application framework," International Journal of Production Research, 2026.

[12] B. Li, K. Mellou, B. Zhang, J. Pathuri, and I. Menache, "Large language models for supply chain optimization," 2023.

[13] K. Yoshizato, K. Shimizu, R. Higa, and T. Otsuka, "AI agent systems for supply chains: Structured decision prompts and memory retrieval," 2026, extended abstract at AAMAS 2026.

[14] Y. Qi, J. Yin, J. Zhang, D. Geng, Z. Chen, H. Hu, W. Qi, and Z.-J. M. Shen, "Leveraging LLM-based agents for intelligent supply chain planning," 2025.

[15] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in International Conference on Learning Representations (ICLR), 2023.

[16] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in Advances in Neural Information Processing Systems (NeurIPS), 2023.

[17] K.-H. Huang et al., "CRMArena: Understanding the capacity of LLM agents to perform professional CRM tasks in realistic environments," in Proceedings of NAACL, 2025.

[18] A. Drouin et al., "WorkArena: How capable are web agents at solving common knowledge work tasks?" in International Conference on Machine Learning (ICML), 2024.

[19] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A benchmark for tool-agent-user interaction in real-world domains," 2024.

[20] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in International Conference on Learning Representations (ICLR), 2024.

[21] S. Dhuliawala, M. Komeili, J. Xu, R. Raileanu, X. Li, A. Celikyilmaz, and J. Weston, "Chain-of-verification reduces hallucination in large language models," in Findings of the Association for Computational Linguistics: ACL 2024, 2024, pp. 3563–3578.
[22] S. Min et al., "FActScore: Fine-grained atomic evaluation of factual precision in long form text generation," in Proceedings of EMNLP, 2023, pp. 12 076–12 100.
[23] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAs: Automated evaluation of retrieval augmented generation," in Proceedings of EACL: System Demonstrations, 2024, pp. 150–158.
[24] B. Wen, J. Yao, S. Feng, C. Xu, Y. Tsvetkov, B. Howe, and L. L. Wang, "Know your limits: A survey of abstention in large language models," Transactions of the Association for Computational Linguistics, vol. 13, pp. 529–556, 2025.
[25] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," 2023.
[26] E. Debenedetti et al., "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track, 2024.
[27] Y. Lu, S. Yang et al., "Proactive agent: Shifting LLM agents from reactive responses to active assistance," in International Conference on Learning Representations (ICLR), 2025.
[28] G. Sahu et al., "InsightBench: Evaluating business analytics agents through multi-step insight generation," in International Conference on Learning Representations (ICLR), 2025.
[29] S. Amershi, D. Weld, M. Vorvoreanu, A. Fourney, B. Nushi, P. Collisson, J. Suh, S. Iqbal, P. N. Bennett, K. Inkpen, J. Teevan, R. Kikin-Gil, and E. Horvitz, "Guidelines for human-AI interaction," in Proceedings of the CHI Conference on Human Factors in Computing Systems, 2019.
[30] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," Journal of the American Statistical Association, vol. 22, no. 158, pp. 209–212, 1927.